# Two simple heptagons meet in at most 38 points: what the machine checked, and what it did not

An audit document for a reader who does not read Lean

24 September 2026

**In one paragraph.** A Lean 4 development in this repository contains a machine-checked theorem: *if two simple heptagons in the plane have finitely many boundary intersection points and share no vertex, their boundaries meet in at most* 38 *points, and* 38 *is attained.* The proof has been checked by Lean's kernel; it uses no `sorry`, no axiom of this project's own, and no `native_decide`. It is **not** a proof that $f(7,7) = 38$: the "share no vertex" hypothesis is the conclusion of §9.1 of Ackerman–Keszegh–Rote, which is assumed here, not proved. This document is written so that you can confirm the first sentence and the last one without reading Lean and without running anything.

## 0. Why you only need to read this document

You have said that the mathematics now rests on the code. The natural next question — "who has read the code?" — has an uncomfortable answer: nobody outside this AI-assisted process has read or run any of it. This section is an argument that for most of the code, that does not matter, and a precise account of the part where it does.

**Lean is a proof checker, not a programming language you must trust.** A Lean file is not an argument addressed to a reader. It is a sequence of instructions for constructing a formal proof object, which is then checked, term by term, against a small fixed set of inference rules — the *kernel*. If the file compiles, a proof object exists and the kernel has verified it. If a step is wrong, it does not compile. There is no "looks right to me" in between.

The consequence is the one that should interest you: **nobody reads the proof files.** Not the author, not a Lean expert, not a referee. This development is 1,779 `.lean` files and roughly 1.6 million lines, all but a few dozen files being machine-generated case enumerations. Reading them would take years and would establish nothing the kernel has not already established more reliably. A Lean expert brought in to audit this repository would not read the proofs either. He would do what §§2–6 of this document do.

**What a human must read is the statement.** A proof checker verifies that a proof proves *its own statement*. It has no opinion about whether that statement is the one you meant. Everything that can go wrong that a machine cannot catch lives in the definitions: if `points9` means the wrong set, or `SimpleClosed` secretly assumes general position, then a flawless kernel-checked proof establishes something you did not ask for.

That surface is small. It is 16 declarations, fewer than sixty lines in total, and the seven that carry the content are printed in §2 beside the mathematics they are supposed to say. **That table is the thing to read.** It is the one place where this could be wrong in a way no machine will find, and reading it takes ten minutes.

**The three loopholes, named.** There are exactly three ways to get something past the kernel, and all three are visible:

- `sorry` — a placeholder that stands in for a missing proof. A file containing `sorry` still compiles; Lean emits a *warning*, not an error, and the build exits 0. This is the dangerous one, because a build log can say "success" over a hole.
- A declared `axiom` — a statement asserted without proof. This also compiles cleanly, with no warning at all.
- `native_decide` — a tactic that closes a goal by compiling it to machine code and running it. It is not a hole and not an axiom; it moves the work out of the 5,000-line kernel and into the whole Lean compiler and the C toolchain beneath it. Vastly more code to trust.

All three are caught by one command. Lean's `#print axioms` on a theorem lists the unproved assumptions its proof depends on — *transitively*, through every lemma it uses. A `sorry` three lemmas down appears in the headline theorem's own axiom list as `sorryAx`; a project axiom appears by name; `native_decide` appears as `Lean.ofReduceBool`. Nothing can be buried. The expected output for this development is exactly

```
[propext, Classical.choice, Quot.sound]
```

which are Lean's own three foundational axioms — propositional extensionality, choice, and quotient soundness — the standard basis of ordinary classical mathematics, shared with all of Mathlib. Any fourth entry is a finding.

**And none of it has to be taken from this machine.** Everything above would still be one party's report about their own computer. It is not: the repository rebuilds itself on GitHub's machines from a fresh clone, runs the axiom check and the sabotage tests, and publishes the result as a pass or a fail that the author cannot forge. That run has happened and it passed, reproducing the figures in §4 and §5 exactly. §6 gives the run, what it covers, and — as importantly — what it does not.

**What is left on trust when you finish this document.** Three things, and they are worth stating plainly because they are all that is left:

1. **Lean's kernel** — that the proof checker itself is correct. It is a small, heavily scrutinised program with independent reimplementations; this is the assumption every formalisation in the literature makes.
2. **That the statement in §2 is the intended one.** You judge this yourself, by reading the table. Nobody can do it for you.
3. **AKR §9.1** — an external lemma, used as a hypothesis, discussed in §7.

Compare this with an ordinary paper. There, a referee must check the definitions *and* follow every step of the argument, and the second job is the one that takes months and the one referees are known to get wrong. Here the second job has been done by a machine. The residual trust is strictly smaller — item 2 above is the referee's first job, unchanged, and items 1 and 3 are new but sharply bounded. That is the whole claim. It is not a claim that formalisation is infallible.

## 1. The claim, in ordinary mathematics

**Proved.** Let $P$ and $Q$ be simple closed heptagons in the plane — seven vertices each, sides $[P_i, P_{i+1}]$ and $[Q_j, Q_{j+1}]$ for $i, j \in \mathbb{Z}/7$, each polygon a closed curve that does not cross itself. Suppose $\partial P \cap \partial Q$ is finite. Then $\#(\partial P \cap \partial Q) \leq 38$, and there is a pair for which the count is exactly 38; so 38 is the maximum.

**Assumed, not proved.** That no vertex of $P$ is equal to a vertex of $Q$. This is a hypothesis of the theorem, written out explicitly; it is the conclusion of §9.1 of Ackerman, Keszegh and Rote, which is not formalised here.

Two remarks on what is *not* assumed, because they are where a formalisation usually cheats.

*No general position.* Three collinear vertices are allowed. A straight vertex — where two consecutive sides are collinear — is allowed. A vertex of one polygon lying in the *interior* of a side of the other is allowed, and is the case AKR §9.2 removes by a perturbation argument; here it is proved, not assumed away. The only degeneracy excluded is vertex-equals-vertex.

*Points, not crossings.* The quantity bounded is the cardinality of a set of points of the plane. A tangential touch that does not cross counts as one point; a point where three sides concur counts once, not twice. This is the harder thing to bound and the thing the question actually asks.

## 2. The statement, beside the mathematics

This is the part to read. The left column is the claim in ordinary mathematical English; the right column is the Lean source, quoted verbatim by `docs/audit/extract_lean.py` from the files and line numbers shown. **Your task is to decide whether the two columns say the same thing.** Nothing else in this document depends on your knowing any Lean: the notation you need is $\forall$, $\wedge$ (and), $\to$ (implies), $\cup$, $\cap$, `Fin 7` $= \mathbb{Z}/7$ with wraparound on `+`, and `segment ℝ a b` $= [a, b]$, the closed segment.

| the mathematics | the Lean, verbatim |
|---|---|
| **(1)** $P$ is a closed heptagon that does not cross itself: no side is a point, non-adjacent sides are disjoint, and two consecutive sides meet exactly in the vertex they share. | `def SimpleClosed (W : Fin 7 → ℝ × ℝ) : Prop :=`<br>`  (∀ i : Fin 7, W i ≠ W (i + 1)) ∧`<br>`  (∀ i j : Fin 7, ¬ LowerBound.Adjacent i j →`<br>`  Disjoint (segment ℝ (W i) (W (i + 1))) (segment ℝ (W j) (W (j + 1)))) ∧`<br>`  (∀ i : Fin 7,`<br>`  segment ℝ (W i) (W (i + 1)) ∩ segment ℝ (W (i + 1)) (W (i + 2))`<br>`  = {W (i + 1)})`<br>`lean/AKR9.lean:71`<br><br>*Note.* This is simplicity *as a curve* and nothing more. Three collinear vertices are deliberately allowed — a straight vertex is not excluded here. `Adjacent` is cyclic adjacency on $\mathbb{Z}/7$. |
| **(2)** The object under study is an ordered pair $(P, Q)$ of such heptagons in the plane. Nothing else is assumed about it. | `structure Pair9 where`<br>`  /-- The fourteen vertices: `P` is `0..6`, `Q` is `7..13`. -/`<br>`  V : Fin 14 → ℝ × ℝ`<br>`  simpleP : SimpleClosed (fun i : Fin 7 => V (pP i))`<br>`  simpleQ : SimpleClosed (fun i : Fin 7 => V (pQ i))`<br>`lean/AKR9.lean:81`<br><br>*Note.* The fourteen vertices are one function `V`; `pP i` $= i$ and `pQ i` $= 7 + i$ split them. There is no general-position field, and no non-degeneracy field. The two `simple` fields are the whole hypothesis. |

**(3)** $\partial P = \bigcup_{i \in \mathbb{Z}/7} [P_i, P_{i+1}]$, the union of the seven closed sides; likewise $\partial Q$.

```
def bdryP9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ i : Fin 7, sideP9 R i
```

lean/AKR9.lean:115

*Note.* `segment ℝ a b` is Mathlib's *closed* segment $\{(1-t)a + tb : 0 \le t \le 1\}$, so endpoints are included. `sideP9` is that segment; `bdryQ9` is the same with `pQ`.

---

**(4)** The set being counted is $\partial P \cap \partial Q$: the points of the plane lying on both boundaries.

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R
```

lean/AKR9.lean:119

*Note.* A set of points, not a list of crossings and not a multiset. A point where three sides concur is one element, not two.

---

**(5)** *Assumed:* no vertex of $P$ equals a vertex of $Q$. (This is the conclusion of AKR §9.1; see §7.)

```
def attainedCoincFree : Set ℕ :=
  { n | ∃ R : Pair9, (points9 R).Finite ∧
      (∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) ∧ (points9 R).ncard = n }
```

lean/Pair9Shift.lean:4314

*Note.* The middle clause is the assumption, written out. It forbids *equality* of vertices only — a vertex lying in the interior of the other polygon's side is *not* excluded, and is handled by the proof.

---

**(6)** If $\partial P \cap \partial Q$ is finite and $P, Q$ share no vertex, then $\#(\partial P \cap \partial Q) \le 38$.

```
theorem points9_le_38_of_coincFree (R : Pair9) (hfin : (points9 R).Finite)
    (hcoinc : ∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) :
    (points9 R).ncard ≤ 38 := by
  obtain ⟨Rend, hD, hE, hb⟩ := exists_chain_deg_free _ R hfin hcoinc rfl
  exact points9_le_38_of_joint R Rend hfin hb hD hE
```

lean/Pair9Shift.lean:4223

*Note.* `ncard` is the cardinality of a set as a natural number. `hfin` is the finiteness hypothesis; without it the count is not a number at all (see the last row of §3).

---

**(7)** 38 is attained, and is the largest attained value: it is the maximum, not merely an upper bound.

```
theorem isGreatest_38_of_coincFree : IsGreatest attainedCoincFree 38 := by
  refine ⟨⟨AKR9.certPair9, certPair9_finite, certPair9_coincFree,
    AKR9.certPair9_ncard⟩, ?_⟩
  rintro n ⟨R, hfin, hco, rfl⟩
  exact points9_le_38_of_coincFree R hfin hco
```

lean/Pair9Shift.lean:4324

*Note.* `IsGreatest S a` unfolds to $a \in S \wedge \forall b \in S,\ b \le a$. The membership half is the explicit certificate of §3; the bound half is the row above.

---

**The drift tripwire.** A quotation is only as good as its currency. `extract_lean.py --check` recomputes the SHA-256 of every declaration above and compares it against the lockfile written when this document was generated. If any declaration has been renamed, moved, or altered by one character, it exits 1 and names the declaration. It is run as part of building this PDF. The table above therefore cannot have drifted away from the code without the build failing.

**One thing this table is not.** The repository contains a *second* chain that also ends at the number 38 — `Heptagon.MainTheorem.isGreatest_38`, over integer-coordinate combinatorial configurations — and §5 prints its axiom line alongside these. It is not the theorem above, and it is **not unconditional**: it carries three explicit hypotheses that are exactly the open part of that route, and §5 sets them out. Six axiom lines in §5 must therefore not be read as six unconditional results. **The claim this**

**document makes is the one in the table above**, and nothing in §§1–4 or 6 depends on the other chain.

## 3. The gallery: hand-countable pictures beside the computed count

The definitions in §2 are set-theoretic. This section checks that they compute what you would count by eye. `docs/audit/gallery.py` transcribes the same predicates into Python — `orientR`, `SimpleClosed`, `Adjacent`, `points9`, each annotated with the Lean file and line it came from — draws 10 configurations, and circles and numbers every point of $\partial P \cap \partial Q$. All arithmetic is exact rational arithmetic (`fractions.Fraction`); no floating point is used for any geometric decision. The tool self-tests 33 properties on startup and aborts if any fails, including a sweep confirming its segment-intersection routine agrees with the textbook orientation test on all 6,561 segment pairs over a $3 \times 3$ grid.

**Lead with the certificate.** Figure 1 is `certificate_38`: the configuration read from the file `lower_bound_38.json`, the same object the Lean proof uses as its witness. Its 38 intersection points are circled and numbered 1 through 38. You can count them. That is the entire lower bound, and it needs no trust at all.

| configuration | both simple? | **computed** | you should get | degeneracies present |
|---|---|---|---|---|
| `certificate_38` | yes | 38 | 38 | — |
| `easy_lens_2` | yes | 2 | 2 | — |
| `easy_bars_cross_4` | yes | 4 | 4 | — |
| `easy_slotted_and_bar_8` | yes | 8 | 8 | — |
| `easy_turned_pair_14` | yes | 14 | 14 | — |
| `awkward_vertex_in_edge_interior` | yes | 2 | 2 | no vertex on other: **fails**; not general position |
| `awkward_three_collinear_in_P` | yes | 4 | 4 | not general position |
| `awkward_shared_vertex` | yes | 2 | 2 | no vertex on other: **fails**; not general position |
| `awkward_tangential_touch` | yes | 1 | 1 | no vertex on other: **fails**; not general position |
| `awkward_overlapping_sides_infinite` | yes | INFINITE | infinite | no vertex on other: **fails**; not general position |

The five `easy_*` rows are sanity checks with obvious answers (two convex heptagons overlapping in a lens: 2; two crossing bars: 4; a convex heptagon and a turned copy: 14, which is 2 per side and the maximum for two convex heptagons). The five `awkward_*` rows are the ones that matter:

- `awkward_vertex_in_edge_interior` — a vertex of $Q$ lies exactly inside a side of $P$. This is the degeneracy AKR §9.2 removes and this project proves; it is *not* excluded by the theorem's hypotheses, and it is counted correctly.
- `awkward_three_collinear_in_P` — $P_0, P_1, P_2$ are collinear, so $P$'s base is one straight line split into two sides. `SimpleClosed` accepts this, deliberately. Any definition that rejected it would be smuggling in general position.
- `awkward_shared_vertex` — $P_0 = Q_0$. This is exactly the case the theorem assumes away. It is drawn here so you can see what the hypothesis excludes.
- `awkward_tangential_touch` — the boundaries meet at one point without crossing. Count 1: a touch is a point of the intersection. A tool that counted *crossings* would report 0 here and would be answering a different question.
- `awkward_overlapping_sides_infinite` — a side of $P$ and a side of $Q$ are collinear and overlap in a whole sub-segment, so $\partial P \cap \partial Q$ is genuinely infinite. The tool reports INFINITE and refuses to

produce a number. This is the honest failure mode, and it is why the theorem carries a finiteness hypothesis: without it the count is not a natural number and `ncard` would silently return 0.

Each configuration carries an expected count written down in advance; a mismatch, or a configuration whose polygons fail `SimpleClosed`, is reported as a discrepancy and the tool exits non-zero. It exits 0.

The 10 figures follow. In each, $P$ is solid blue, $Q$ is dashed red, and every point of $\partial P \cap \partial Q$ is circled and numbered.

## 4. Sabotage: does the checker ever say no?

A verification that cannot fail proves nothing. If a check passes on this repository, the first question is whether it would also pass on a broken one. The repository answers this with controlled mutations: take a throwaway copy, introduce one specific defect, and require that the check rejects it *and names the right reason*. A control copy with no defect must still be accepted, so that "rejects everything" is not mistaken for rigour.

**L4 first, because it is the one that is not about broken proofs.** Every other test below breaks a *proof*. L4 breaks a *definition*. The mutation changes one character in `lean/AKR9.lean`:

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R
```

*becomes*

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∪ bdryQ9 R
```

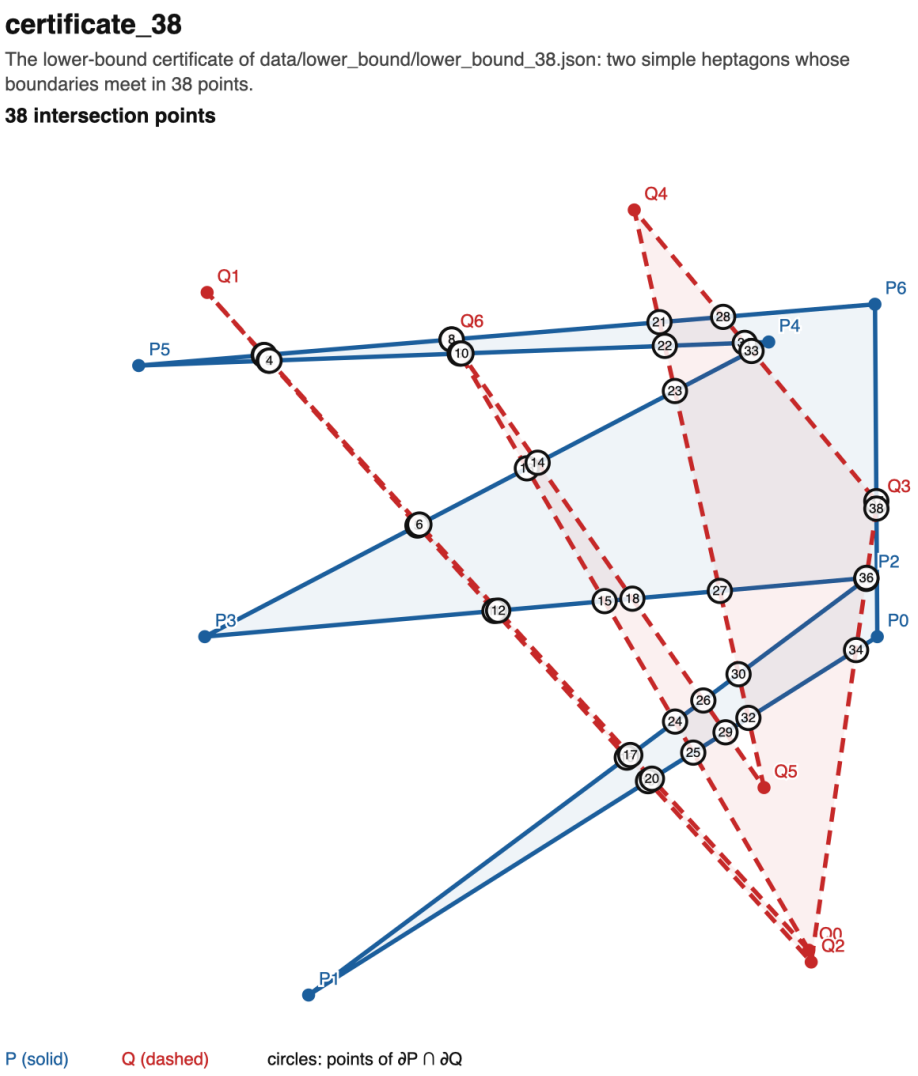
**Not one character of proof text is touched.** There is no `sorry`, no axiom, no `native_decide`; the mutated file is a perfectly legitimate Lean development about the union of two boundaries instead of their intersection — a set that is obviously not of size at most 38. The requirement is that the build *fail*. It does: that is case L4 of the run recorded below.

Without this test, "Lean proved it" would mean only that *something* was proved. With it, the theorem is pinned to the definitions you read in §2: change what `points9` means and the proofs must stop type-checking. That is what shows the definitions are load-bearing and not decoration — and it is what the run below shows. A reader who spends ten minutes on the 7 rows of §2 is therefore checking text that genuinely determines the result, not prose sitting above proofs that would compile whatever it said.

**The three groups.** `scripts/negative-tests.sh` has three groups. Group N mutates the evidence files (certificates, hashes, refutation proofs). Group A mutates the *axiom log* and drives the repository's axiom checker over it — these need no Lean at all, because the checker is a log parser and can be tested the way any parser is tested. Group L is the four Lean-level mutations, L1–L4; they require a real `lake build` of each mutated copy and are opt-in behind a `--lean` flag.

**Groups A and L were both run while this document was written**, and both transcripts are committed. The two tables below are copied line for line from those runs, not transcribed.

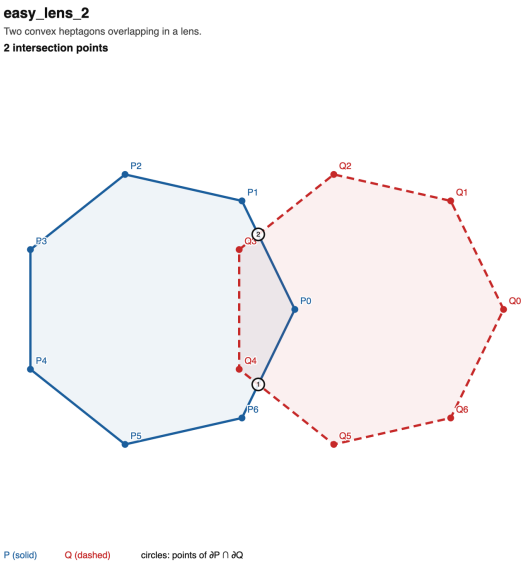
One thing to know before reading them. `negative-tests.sh` can print five verdicts — *accepted*, *caught*, NOT CAUGHT, NOT APPLIED, WRONGLY REJECTED — and the generator that builds these tables renders all five. **An absence of failures below is therefore a fact about the runs, not a limitation of the table.** That distinction is not rhetorical: an earlier version of the generator recognised only three of the five verdicts and would have silently dropped a failing row. That is the same defect as a checker that cannot fail — the very thing this section exists to rule out — and it was found and fixed before this document was built.




`certificate_38`

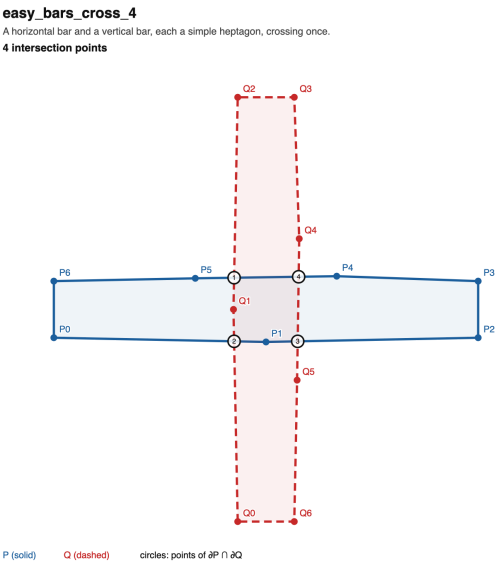
computed count: **38**

Figure 1: The lower-bound certificate. Every point of $\partial P \cap \partial Q$ is circled and numbered; there are **38**. This is the same configuration the Lean proof uses as its witness, read from the same JSON file.



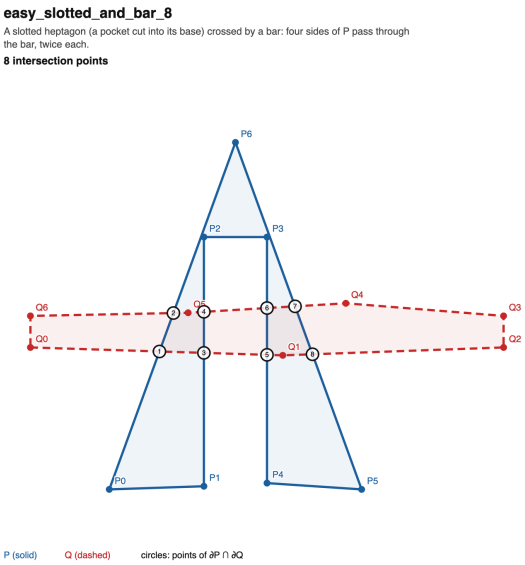

`easy_lens_2`

computed count: **2**



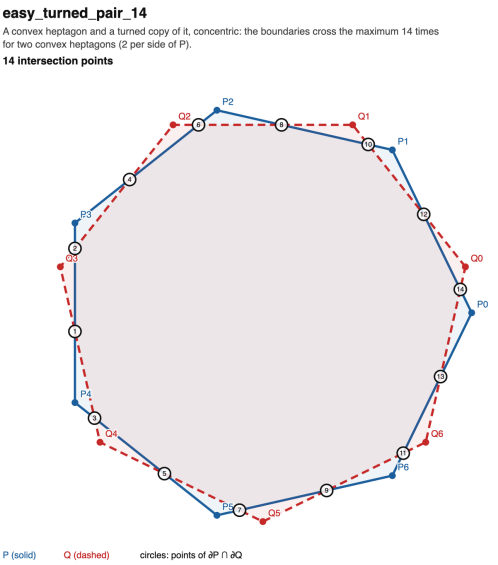

`easy_bars_cross_4`

computed count: **4**




`easy_slotted_and_bar_8`

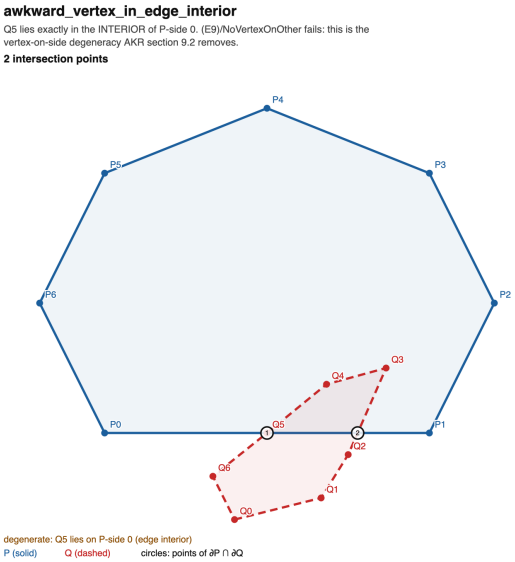
computed count: **8**




`easy_turned_pair_14`

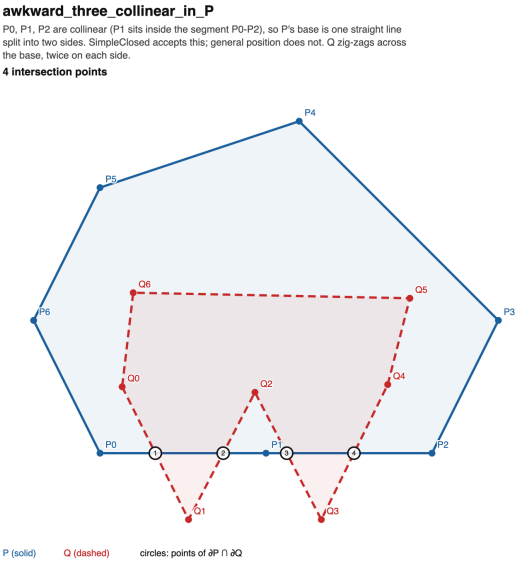
computed count: **14**

Figure 2: Sanity checks with answers you can see at a glance.



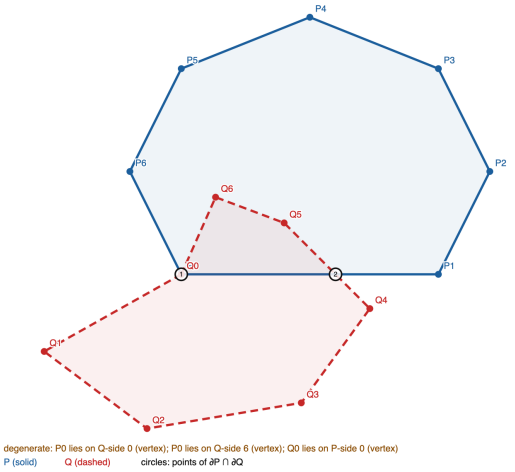

awkward_vertex_in_edge_interior

computed count: **2**



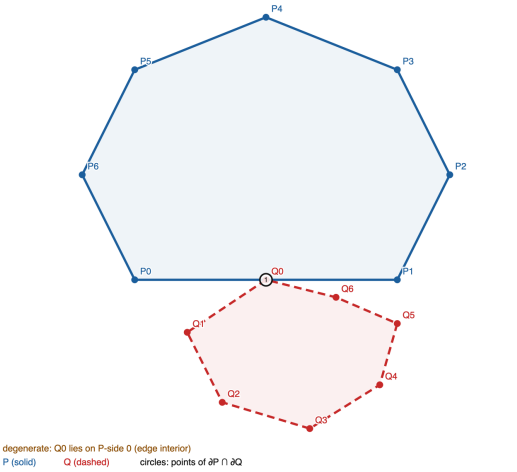

awkward_three_collinear_in_P

computed count: **4**




awkward_shared_vertex

computed count: **2**




awkward_tangential_touch

computed count: **1**

Figure 3: Degeneracies that the theorem's hypotheses *allow*, plus the one they exclude (`awkward_shared_vertex`).

| case | the mutation | verdict |
|---|---|---|
| A1 | complete clean log (control) | accepted |
| | *nothing — a complete, clean #print axioms log* | |
| A2 | wrapped axiom list (control) | accepted |
| | *nothing — but one axiom list is wrapped across three lines* | |
| A3 | sorryAx in a headline theorem | **caught** |
| | *a headline theorem's axiom list contains sorryAx* | |
| A4 | headline lines absent | **caught** |
| | *a log whose headline modules were never built: clean, but they never printed* | |
| A5 | disallowed axiom (ofReduceBool) | **caught** |
| | *a headline theorem depends on Lean.ofReduceBool (what native_decide leaves)* | |
| A6 | project-declared axiom | **caught** |
| | *a headline theorem depends on a project-declared axiom* | |
| A7 | no axiom output at all | **caught** |
| | *a build log with no #print axioms output whatsoever* | |
| A8 | disallowed axiom in wrapped tail | **caught** |
| | *a disallowed axiom hidden on the wrapped continuation line* | |

**RESULT: 8 passed, 0 failed** (bash scripts/negative-tests.sh --groups A, exit 0)

**Group L was run, and the transcript is committed.** The four Lean-level mutations — a planted `sorry` (L1), a declared and used `axiom` (L2), a `native_decide` (L3), and the changed definition described above (L4) — each plant one defect in a copy-on-write clone of this repository, build the clone with a real `lake build`, and require the verifier to *reject* it. The run is recorded at `results/validation/2026-09-24_negative_tests_L.log`.

**date** 2026-09-25T01:37:04Z **uname** Darwin 24.6.0 arm64 **cmd** NEGTESTS_LAKE_DIR=$PWD/lean/.lake bash scripts/negative-tests.sh –lean –groups L

| case | the mutation | verdict |
|---|---|---|
| L1 | sorry in a dependency | **caught** |
| | *a sorry planted three lemmas below a headline theorem; the log also shows `sorryAx`, so the mechanism under test is what fired* | |
| L2 | declared axiom, used | **caught** |
| | *an axiom was declared and used beneath a headline theorem; the log also shows `points9_eq_range_ax`, so the mechanism under test is what fired* | |
| L3 | native_decide | **caught** |
| | *a proof was closed by native_decide (compiler, not kernel); the log also shows `ofReduceBool`/`native_decide`, so the mechanism under test is what fired* | |
| L4 | definition changed (intersect->union) | **caught** |
| | *points9 redefined from an INTERSECTION to a UNION (no proof touched)* | |

**RESULT: 4 passed, 0 failed** (bash scripts/negative-tests.sh --lean --groups L)

Every one of the four mutations was rejected by a real build of a real mutated copy. L4 is the one that bears on §2 directly: not one character of proof text was altered, only the *definition*, and the build still had to fail — and did. The definitions you read in §2 are therefore load-bearing, not decoration.

## 5. What the machine checked, and how to re-run it

```
cd lean && lake build Headline
```

`lean/Headline.lean` declares nothing. It imports the four roots of the live chain — `MainTheorem`, `RealUpperBound`, `AKR9`, `Pair9Shift` — and asks for the axiom lists of five declarations:

```
#print axioms Heptagon.MainTheorem.isGreatest_38
#print axioms Heptagon.RealUpperBound.isGreatest_38_real
#print axioms Heptagon.AKR9.isGreatest_38_of_E9
#print axioms Heptagon.Pair9Shift.isGreatest_38_of_coincFree
#print axioms Heptagon.Pair9Shift.points9_le_38_of_coincFree
```

Building that one target builds exactly the transitive closure of the four roots — **331 modules** — and nothing else. The roots are listed weakest-hypothesis-last; the last two printed declarations are the ones §2 quotes. (A bare `lake build` does *not* build them: the project's `defaultTargets` deliberately omits the §9 modules, so that rebuilding them cannot invalidate the 1,080 enumeration shards. `Headline` exists to close that gap with one command.)

**The transcript of that command is committed.** A full transcript of that build is in the repository, at `results/validation/2026-09-24_headline_build.log`: 83,020 lines, opening with a provenance header that records the date (2026-09-25T01:36:38Z), the machine, the exact Lean version, and the exact command. Read out of that file:

- it ends with `Build completed successfully (3,344 jobs).`;
- it carries 408 `depends on axioms` lines, and across all of them exactly 3 distinct axioms appear: `propext`, `Classical.choice` and `Quot.sound`. There is no fourth entry anywhere in the file;
- `sorryAx` occurs 0 times in it, and `Lean.ofReduceBool` — what `native_decide` would leave — 0 times;
- each of the six theorems the repository's checker demands printed an axiom line of its own. Five of the `#print axioms` *directives* that produce those lines sit in `lean/Headline.lean` at lines 45–49. The sixth, for `points_le_38_of_no_vertex_on_boundary`, is a directive at `lean/AKR9.lean:420` — the *theorem* itself is stated at `lean/AKR9.lean:354` — which is why that one axiom line appears about a thousand lines earlier in the log than the other five.

You need not accept my reading of it. The repository's axiom checker is a log parser that needs no Lean toolchain, and it can be pointed at that file directly:

```
bash scripts/verify-all.sh --axiom-log
results/validation/2026-09-24_headline_build.log
```

It exits 0, and reports 409 declarations parsed, 0 of them using a disallowed axiom, and 6 of 6 headline theorems confirmed. Its full output is committed at `docs/audit/out/axiom_check_headline.log`. The transcript's own provenance header, its six headline axiom lines, its last line, and the checker's verdict on it are reproduced here:

```
# date  : 2026-09-25T01:36:38Z
# uname : Darwin 24.6.0 arm64
# lean  : Lean (version 4.33.1, arm64-apple-darwin24.6.0,
           commit 819816b2e0a3bf405af45ae5c7af2491d8f5bee6, Release)
# cmd   : cd lean && lake build Headline
  …
info: AKR9.lean:420:0: 'Heptagon.AKR9.points_le_38_of_no_vertex_on_boundary'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:45:0: 'Heptagon.MainTheorem.isGreatest_38'
    depends on axioms: [propext, Quot.sound]
info: Headline.lean:46:0: 'Heptagon.RealUpperBound.isGreatest_38_real'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:47:0: 'Heptagon.AKR9.isGreatest_38_of_E9'
    depends on axioms: [propext, Classical.choice, Quot.sound]
info: Headline.lean:48:0: 'Heptagon.Pair9Shift.isGreatest_38_of_coincFree'
    depends on axioms: [propext, Classical.choice, Quot.sound]
```

```
info: Headline.lean:49:0: 'Heptagon.Pair9Shift.points9_le_38_of_coincFree'
    depends on axioms: [propext, Classical.choice, Quot.sound]
Build completed successfully (3344 jobs).
```

Quoted from `results/validation/2026-09-24_headline_build.log` (83,020 lines): the provenance header, the six headline axiom lines (log lines 82448, 83015, 83016, 83017, 83018, 83019), and the final line 83020.

The repository's own axiom checker, run over that transcript (the command boxed above; exit 0) reports:

```
'depends on axioms' lines found          : 408
declarations parsed                      : 409
declarations using a disallowed axiom    : 0
occurrences of sorryAx                   : 0
distinct axioms                          : 3
headline theorems confirmed              : 6 of 6
```

*AXIOM CHECK: CLEAN — 409 declaration(s) reported their axioms; every one of the 3 distinct axioms used is among propext, Classical.choice, Quot.sound; sorryAx appears nowhere; all 6 headline theorems confirmed.*

**Six axiom lines are not six unconditional theorems.** They all carry the number 38 and they are not the same claim, so they are worth separating before you read the list above.

Five of the six are the chain §2 quotes: `isGreatest_38_real`, `isGreatest_38_of_E9`, `points_le_38_of_no_vertex_on_boundary`, `points9_le_38_of_coincFree` and `isGreatest_38_of_coincFree`. They assume nothing beyond the hypotheses written into their own statements — which are exactly the ones you read in §2: finiteness, and no shared vertex. The counting bound they all rest on, `RawConfig.raw_crossPairs_card_le_38` (`lean/RawConfig.lean:231`), takes a bare pair of heptagons and *no general-position hypothesis at all*.

**The sixth is a different animal.** `MainTheorem.isGreatest_38` (`lean/MainTheorem.lean:88`) is about a different object — integer-coordinate, orientation-level `Config`s — and it is *conditional*. It carries three explicit hypotheses which are precisely what is still open on that route: that every configuration has an even crossing count, that none attains 42, and that none attains 40, the last of these being the entire 13,003,620-graph enumeration. Its own doc-comment (`lean/MainTheorem.lean:45--48`) puts it plainly:

*"So the honest reading of `isGreatest_38` below is: '38 is the maximum crossing count over integer-coordinate, orientation-level heptagon configurations, GIVEN the listed hypotheses.' Getting from there to `f(7,7) = 38` needs the Jordan-curve step and the general-position reduction, both external."*

Note in the transcript above that this one has the *shortest* axiom list of the six — `[propext, Quot.sound]`, with no `Classical.choice`. That is a consequence of its being conditional, not a sign that it is the strongest of them: a hypothesis you have not discharged costs no axioms. **Nothing in this document rests on it.** The claim in §1, the statement in §2 and the non-vacuity of §6 are all the other route.

**Two figures in this section that the transcript does *not* establish.** It records no elapsed time, so the **11 s** warm-build figure is still only the repository's own assertion (`lean/README.md`) and you should read it as that. And the log counts *jobs*, not modules: the 3,344 above is a job count, while the **331 modules** figure is corroborated separately — including by a transitive-closure computation recorded in `docs/verification-status.md` — and the two must not be conflated.

**A correction, and what the axiom check really guards against.** An earlier draft of this section asserted that `#print axioms` prints only when a module is actually recompiled, so that a warm rebuild "replays nothing and therefore prints nothing" and only a cold build could produce this evidence. **That was wrong**, and the committed transcript is what demonstrates it: all 271 of its progress lines read `Replayed` and not one reads `Built` — it is a fully warm run — and it nevertheless carries every one of the 408 axiom lines, because Lake replays the cached messages of the modules it does not recompile.

The trap the checker really guards against is a different one, and a worse one: a log in which the headline theorems never appear *at all*. That is what a bare `lake build` produces here, because `defaultTargets` omits the §9 modules, so those theorems are never part of the build and never print. Such a log exits 0 and looks clean while establishing nothing whatever about the statements you care about. Test A4 in §4 is exactly that

case and the checker rejects it. Its requirement is not that the build was cold; it is that each of the six named theorems printed an axiom line of its own.

**If you did want to run it.** A fresh clone must first download Mathlib (the Lean mathematical library, pinned here at `v4.33.1`) before any of this project's own code compiles. The repository records about **1.4 GB** left in `lean/.lake` after a cold build, and flags that figure as measured before later modules were added and not re-measured since, so treat it as a lower bound. That download, not this development, dominates the cost of a first build. You are not expected to do any of this; the point of §§2–4 is that you do not have to.

## 6. Someone else already ran it

Everything in §§2–6 was produced on the author's computer. You are entitled to ask why any of it should be believed, and the answer cannot be more output from the same machine.

So the repository builds itself on GitHub's. On every push to the main branch, GitHub clones the repository onto a machine nobody involved here owns or can log into, installs the pinned Lean toolchain, compiles the headline chain *from source*, and runs the axiom check and the group-A sabotage tests of §4. If any step fails, the run goes red and stays red in public view. The green mark is not something the author can produce by hand.

**First run: passed.** 25 September 2026, 40 min 54 s

`github.com/NewYorkImperialist/heptagon-intersections/actions/runs/36086080499`

commit `b78f372`, runner `ubuntu-latest`, 2 cores, 7.8 GiB

What that run reported, quoted from `docs/audit/out/ci_run.log`:

```
lean/lean-toolchain pins: leanprover/lean4:v4.33.1
Build completed successfully (3344 jobs).
OK: 0 occurrences of sorryAx
   axiom-check: declarations parsed              : 409
   axiom-check: distinct axioms                  : 3
   axiom-check: headline theorems confirmed      : 6 of 6
   AXIOM CHECK: CLEAN
RESULT: 8 passed, 0 failed          (the group-A sabotage tests)
```

These are the same figures §5 and §4 report from the author's machine — the same 3,344 jobs, the same 409 declarations, the same six theorems, the same three axioms. That is the point: an independent machine, starting from nothing but the repository, reached them again.

**The build is genuinely from source, and this is load-bearing.** Lake will happily restore a previous build and *replay* its stored messages, which look identical to freshly produced ones. A workflow that cached the build tree would therefore go green while re-checking nothing — its axiom lines would be text unpacked from an archive, and the tick would attest GitHub's ability to open a tarball. The workflow deliberately does not cache `lean/.lake/build`, and the run above shows **346 modules *Built* and 0 *Replayed***: every one of this project's modules was elaborated by that runner's kernel. (Mathlib, the external library, *is* fetched prebuilt. Caching a dependency is ordinary; caching the thing you claim to have verified is not.)

**What the green mark does not cover.** Two things, so that it is not read for more than it says. It does not run the *Lean* mutation tests L1–L4 of §4 — each rebuilds a deliberately broken copy of the repository and would exceed the six-hour limit, so that job is opt-in and was skipped; their evidence is the committed transcript instead. And it builds the 332-module headline closure, not all 1,779 declared libraries, most of which are generated data modules from an abandoned approach that nothing current imports. Neither omission touches the six theorems.

**The repository is private.** It is held back deliberately, to establish priority before the result is public, so the run above is not visible to the world. A named collaborator can see it, and the intended reader of this document is expected to be one.

## 7. Non-vacuity: the theorem is not true for a boring reason

A conditional theorem whose hypotheses are contradictory is vacuously true and worthless. An upper bound nobody attains is not a maximum. Both failure modes are closed explicitly at the end of `lean/Pair9Shift.lean`, by a single bundled declaration, `nonVacuity`, in the `Heptagon.Pair9Shift` namespace.

`lean/Pair9Shift.lean:5296`

```
theorem nonVacuity :
    Nonempty AKR9.Pair9
    ∧ (points9 AKR9.certPair9).Nonempty
    ∧ (points9 AKR9.certPair9).Finite
    ∧ (∀ v j : Fin 7, AKR9.certPair9.V (pP v) ≠ AKR9.certPair9.V (pQ j))
    ∧ AKR9.NoVertexOnOther AKR9.certPair9
    ∧ (points9 AKR9.certPair9).ncard = 38
    ∧ ¬ (points9 AKR9.certPair9).ncard ≤ 37
    ∧ (38 : ℕ) ∈ attainedCoincFree
    ∧ (39 : ℕ) ∉ attainedCoincFree
    ∧ IsGreatest attainedCoincFree 38 :=
  ⟨pair9_nonempty, certPair9_points9_nonempty, certPair9_finite,
   certPair9_coincFree, certPair9_noVertexOnOther, AKR9.certPair9_ncard,
   certPair9_not_le_37, mem_attainedCoincFree_38, not_mem_attainedCoincFree_39,
   isGreatest_38_of_coincFree⟩
```

Ten clauses, in order:

**(1)** `Pair9` is inhabited: the two `SimpleClosed` hypotheses of §2 are satisfiable at all. Without this the whole theory could be about an empty type.

**(2)** The witness's intersection set is non-empty.

**(3)** It is *finite* — so `ncard` is the real cardinality. On an infinite set `ncard` returns 0, which would make the bound $\leq 38$ true and meaningless; this clause closes that door. (Clause (6) already implies it, for exactly that reason; it is stated separately for the reader.)

**(4)** The witness satisfies `hcoinc`: it shares no vertex. The hypothesis of the theorem is not contradictory.

**(5)** The witness also satisfies the *stronger* condition `NoVertexOnOther` — no vertex anywhere on the other boundary.

**(6)** Its count is exactly 38: **the bound is attained**.

**(7)** $\leq 37$ is *false* there: **the bound is not slack**. One less is provably wrong, so 38 is the right constant and not an artefact of a lossy estimate. (Immediate from (6); stated because it is the thing a reader wants to see said.)

**(8)** 38 is an attained value.

**(9)** 39 is *not* attained.

**(10)** 38 is the greatest attained value.

These are ten clauses, not ten logically independent facts: (6) is the sharpest of them and already implies (3) and (7). The content is not the number of clauses but the fact that **(2)–(7) all hold of the *same* configuration**, `AKR9.certPair9`. Checking hypotheses one at a time would not rule out a theorem whose hypotheses are each satisfiable but jointly contradictory; a single shared witness does. That is precisely the failure mode an absent human reviewer would be looking for, and the declaration's own doc-comment says so.

A companion records that the maximum is *unique* — any $n$ that is a greatest attained value equals 38:

`lean/Pair9Shift.lean:5272`

```
theorem isGreatest_attainedCoincFree_unique {n : ℕ}
    (h : IsGreatest attainedCoincFree n) : n = 38 :=
  h.unique isGreatest_38_of_coincFree
```

## 8. What is not proved

This section is the reason to believe the rest.

**$f(7,7) = 38$ is not a Lean theorem.** The repository's own status document says so in those words — "**`f(7,7) = 38` is still not a Lean theorem**" — and the doc-comment on the theorem itself repeats it: "*This is not `f(7,7) = 38`, and it is not all of AKR §9.*" What is proved is the statement in §2, with the no-shared-vertex hypothesis explicit, and with finiteness of the intersection assumed rather than derived. That second hypothesis is not a technicality: the last row of §3 is a genuine configuration where it fails.

**AKR §9.1 is assumed.** The reference throughout is E. Ackerman, B. Keszegh and G. Rote, *An almost optimal bound on the number of intersections of two simple polygons*, Discrete Comput. Geom. **68** (2022) 1049–1077. Their §9 shows that a pair of simple polygons may be perturbed into one where no vertex of either lies on the other's boundary, without decreasing the intersection count. It splits in two. §9.2's half — a vertex in the *interior* of a side — is *proved here*, in Lean, and is no longer imported; that is what `points9_le_38_of_coincFree` accomplishes over its predecessor. §9.1's half — a vertex *equal* to a vertex of the other polygon — is still imported. It is carried as the single explicit hypothesis `hcoinc` rather than baked into a structure, precisely so that a reader can see it and check it against the paper.

As a *statement* it is the kind a geometer grants on sight: coinciding vertices, moved apart by enough to separate them and little enough to disturb nothing else. Its plausibility is not in question. Its *proof* is another matter, and the repository's notes on §9.1 are careful about this: AKR establish it through their Theorem 17 and a dependence-graph argument about which perturbations can be solved simultaneously, which is several pages and not a one-liner. So "routine" here means "believable without checking", not "short". It is nonetheless *not machine-checked here*, and the project has found two genuine defects elsewhere in AKR's published §9 procedure — §9.2's shift is undefined at a straight vertex, and a direction meeting AKR's stated criterion can make the perturbed intersection infinite — so "published" has not been treated as "correct" in this development. Both defects are repairable and both repairs are this project's, not AKR's.

**Nobody outside this process has reviewed any of it.** Not the Lean, not the Python, not the mathematics. This document is the first artifact written to be read by someone outside it. The *proofs* do not need a human reader, for the reason given in §0; the *statement* and this document's claims about it do, and you are that reader.

**What the complete argument therefore is.** Not a single artifact. It is **a machine-checked theorem plus one routine lemma a geometer grants on sight** — where "routine" keeps the meaning fixed two paragraphs above: believable without checking, but roughly three pages of AKR to actually prove, and not short. The repository's status document calls §9.1 "the one remaining external input". That is a real proof, and it is how a great deal of mathematics is actually assembled. But it is two pieces, of two different kinds, and it should be described that way — not as "formally verified".

**And one smaller thing, so that the list is complete.** The gallery in §3 is a Python *transcription* of the Lean predicates, checked against them by hand and annotated with line numbers; it is independent evidence that the definitions compute what you expect, not a proof that the Python and the Lean agree. It does not affect the kernel-checked status of the theorem in §2, but it does affect how much of *this document* you are taking on my word rather than seeing for yourself.

Two further items stood in this list in an earlier draft, and it is better to record their closure than to delete them quietly. That the §5 build numbers were "the project's own assertions with no committed transcript" is no longer true: the transcript is committed at `results/validation/2026-09-24_headline_build.log`, and the repository's axiom checker can be run over it by anyone, with no Lean toolchain. That "group L of the mutation suite has never been run in a way this repository records" is no longer true either: it was run, its transcript is committed, and §4 reports case by case what it found. Both were real gaps. Both are now artifacts you can open yourself, which is the only kind of answer this document is entitled to give.

## A. The statement surface, verbatim and complete

All 16 declarations extracted by `docs/audit/extract_lean.py`, unedited, with file and line. The six that carry the content appear in §2; these are the rest, and the full text of those six.

**pP** — lean/ResidualSymBase.lean:23

```
def pP (i : Fin 7) : Fin 14 := ⟨i.val, by omega⟩
```

**pQ** — lean/ResidualSymBase.lean:26

```
def pQ (i : Fin 7) : Fin 14 := ⟨7 + i.val, by omega⟩
```

**SimpleClosed** — lean/AKR9.lean:71

```
def SimpleClosed (W : Fin 7 → ℝ × ℝ) : Prop :=
  (∀ i : Fin 7, W i ≠ W (i + 1)) ∧
  (∀ i j : Fin 7, ¬ LowerBound.Adjacent i j →
      Disjoint (segment ℝ (W i) (W (i + 1))) (segment ℝ (W j) (W (j + 1)))) ∧
  (∀ i : Fin 7,
      segment ℝ (W i) (W (i + 1)) ∩ segment ℝ (W (i + 1)) (W (i + 2))
        = {W (i + 1)})
```

**Pair9** — lean/AKR9.lean:81

```
structure Pair9 where
  /-- The fourteen vertices: `P` is `0..6`, `Q` is `7..13`. -/
  V : Fin 14 → ℝ × ℝ
  simpleP : SimpleClosed (fun i : Fin 7 => V (pP i))
  simpleQ : SimpleClosed (fun i : Fin 7 => V (pQ i))
```

**NoVertexOnOther** — lean/AKR9.lean:103

```
def NoVertexOnOther (R : Pair9) : Prop :=
  (∀ i j : Fin 7, R.V (pP i) ∉ segment ℝ (R.V (pQ j)) (R.V (pQ (j + 1)))) ∧
  (∀ i j : Fin 7, R.V (pQ j) ∉ segment ℝ (R.V (pP i)) (R.V (pP (i + 1))))
```

**sideP9** — lean/AKR9.lean:109

```
def sideP9 (R : Pair9) (i : Fin 7) : Set (ℝ × ℝ) :=
  segment ℝ (R.V (pP i)) (R.V (pP (i + 1)))
```

**sideQ9** — lean/AKR9.lean:112

```
def sideQ9 (R : Pair9) (j : Fin 7) : Set (ℝ × ℝ) :=
  segment ℝ (R.V (pQ j)) (R.V (pQ (j + 1)))
```

**bdryP9** — lean/AKR9.lean:115

```
def bdryP9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ i : Fin 7, sideP9 R i
```

**bdryQ9** — lean/AKR9.lean:116

```
def bdryQ9 (R : Pair9) : Set (ℝ × ℝ) := ⋃ j : Fin 7, sideQ9 R j
```

**points9** — lean/AKR9.lean:119

```
def points9 (R : Pair9) : Set (ℝ × ℝ) := bdryP9 R ∩ bdryQ9 R
```

**attainedCoincFree** — lean/Pair9Shift.lean:4314

```
def attainedCoincFree : Set ℕ :=
  { n | ∃ R : Pair9, (points9 R).Finite ∧
      (∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) ∧ (points9 R).ncard = n }
```

**points9_le_38_of_coincFree** lean/Pair9Shift.lean:4223

```
theorem points9_le_38_of_coincFree (R : Pair9) (hfin : (points9 R).Finite)
    (hcoinc : ∀ v j : Fin 7, R.V (pP v) ≠ R.V (pQ j)) :
    (points9 R).ncard ≤ 38 := by
  obtain ⟨Rend, hD, hE, hb⟩ := exists_chain_deg_free _ R hfin hcoinc rfl
  exact points9_le_38_of_joint R Rend hfin hb hD hE
```

**isGreatest_38_of_coincFree** lean/Pair9Shift.lean:4324

```
theorem isGreatest_38_of_coincFree : IsGreatest attainedCoincFree 38 := by
  refine ⟨⟨AKR9.certPair9, certPair9_finite, certPair9_coincFree,
    AKR9.certPair9_ncard⟩, ?_⟩
  rintro n ⟨R, hfin, hco, rfl⟩
  exact points9_le_38_of_coincFree R hfin hco
```

**nonVacuity** lean/Pair9Shift.lean:5296

```
theorem nonVacuity :
    Nonempty AKR9.Pair9
    ∧ (points9 AKR9.certPair9).Nonempty
    ∧ (points9 AKR9.certPair9).Finite
    ∧ (∀ v j : Fin 7, AKR9.certPair9.V (pP v) ≠ AKR9.certPair9.V (pQ j))
    ∧ AKR9.NoVertexOnOther AKR9.certPair9
    ∧ (points9 AKR9.certPair9).ncard = 38
    ∧ ¬ (points9 AKR9.certPair9).ncard ≤ 37
    ∧ (38 : ℕ) ∈ attainedCoincFree
    ∧ (39 : ℕ) ∉ attainedCoincFree
    ∧ IsGreatest attainedCoincFree 38 :=
  ⟨pair9_nonempty, certPair9_points9_nonempty, certPair9_finite,
   certPair9_coincFree, certPair9_noVertexOnOther, AKR9.certPair9_ncard,
   certPair9_not_le_37, mem_attainedCoincFree_38, not_mem_attainedCoincFree_39,
   isGreatest_38_of_coincFree⟩
```

**certPair9_not_le_37** lean/Pair9Shift.lean:5252

```
theorem certPair9_not_le_37 : ¬ (points9 AKR9.certPair9).ncard ≤ 37 := by
  intro h
  rw [AKR9.certPair9_ncard] at h
  omega
```

**isGreatest_attainedCoincFree_unique** lean/Pair9Shift.lean:5272

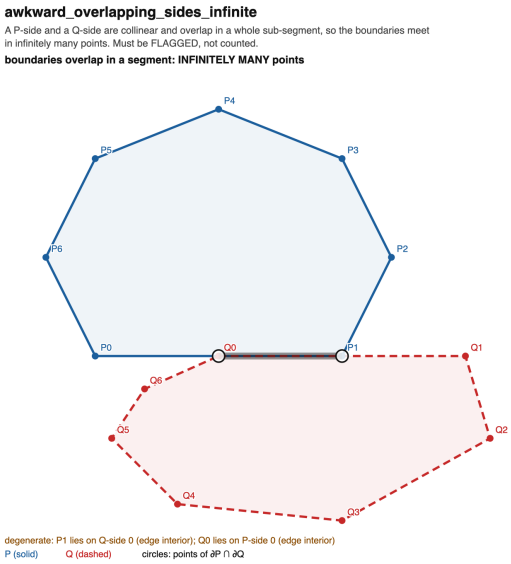
```
theorem isGreatest_attainedCoincFree_unique {n : ℕ}
    (h : IsGreatest attainedCoincFree n) : n = 38 :=
  h.unique isGreatest_38_of_coincFree
```

## B. How this document was produced

```
python3 docs/audit/extract_lean.py    # verbatim Lean   -> out/lean_excerpts.*
python3 docs/audit/gallery.py         # figures, counts -> out/gallery/
for f in docs/audit/out/gallery/*.svg; do            # SVG -> PNG (macOS)
  qlmanage -t -s 1400 -o docs/audit/out/png "$f"; done
bash scripts/negative-tests.sh --groups A \
     > docs/audit/out/negative_tests_A.log           # group A table in 4
bash scripts/negative-tests.sh --lean --groups L \                 # group L in 4
     > results/validation/2026-09-24_negative_tests_L.log
bash scripts/verify-all.sh \                                       # the numbers in 5
     --axiom-log results/validation/2026-09-24_headline_build.log \
```

```
    > docs/audit/out/axiom_check_headline.log
python3 docs/audit/make_tables.py     # the LaTeX fragments
python3 docs/audit/extract_lean.py --check   # drift tripwire; MUST exit 0
cd docs/audit && tectonic audit.tex
```

No Lean in this document was typed by hand; no count in it was computed by hand. The prose in the left column of §2 and in the notes beneath each row is written by a human and is exactly what you are being asked to check.




`awkward_overlapping_sides_infinite`

computed count: **infinite**

Figure 4: The honest failure mode: two sides overlap in a whole sub-segment, so the intersection is genuinely infinite. It is flagged, not miscounted.